

8-mavzu. Takrorlanuvchi jarayonlarga dasturlar tuzish.

Reja

1. Python dasturlash tilida takrorlanuvchi operatorlari.
2. Takrorlanuvchi operatorlar turlari, tuzilishi qo'llanilishi.
3. While operator, for operatori tuzilishi, dasturda qo'llanilishi.
4. Takrorlanuvchi jarayonlarga dasturlar tuzish

Kirish

Dasturlashda ko'plab masalalar bir xil amallarni bir necha bor takrorlashni talab qiladi. Masalan, sonlar yig'indisini hisoblash, massiv elementlarini ko'rib chiqish, ma'lum shart bajarilguncha jarayonni davom ettirish kabi vaziyatlar takrorlanuvchi jarayonlarsiz amalga oshirilmaydi. Agar har bir amal alohida yozilsa, dastur juda uzun, tushunarsiz va xatolarga moyil bo'lib qoladi.

Shu sababli dasturlash tillarida **takrorlanuvchi operatorlar (sikllar)** mavjud. Takrorlanuvchi operatorlar yordamida ma'lum buyruqlar guruhi bir necha marta avtomatik tarzda bajariladi. Python dasturlash tilida takrorlanuvchi jarayonlar juda sodda va tushunarli sintaksis orqali tashkil etiladi.

Mazkur mavzuda Python dasturlash tilida takrorlanuvchi operatorlar, ularning turlari, while va for operatorlarining tuzilishi hamda ular yordamida takrorlanuvchi jarayonlarga dasturlar tuzish masalalari batafsil yoritiladi. Ushbu mavzu keyingi mavzularda break, continue, funksiyalar va massivlar bilan ishlash uchun mustahkam poydevor bo'lib xizmat qiladi.

1. Python dasturlash tilida takrorlanuvchi operatorlar

Takrorlanuvchi jarayon tushunchasi

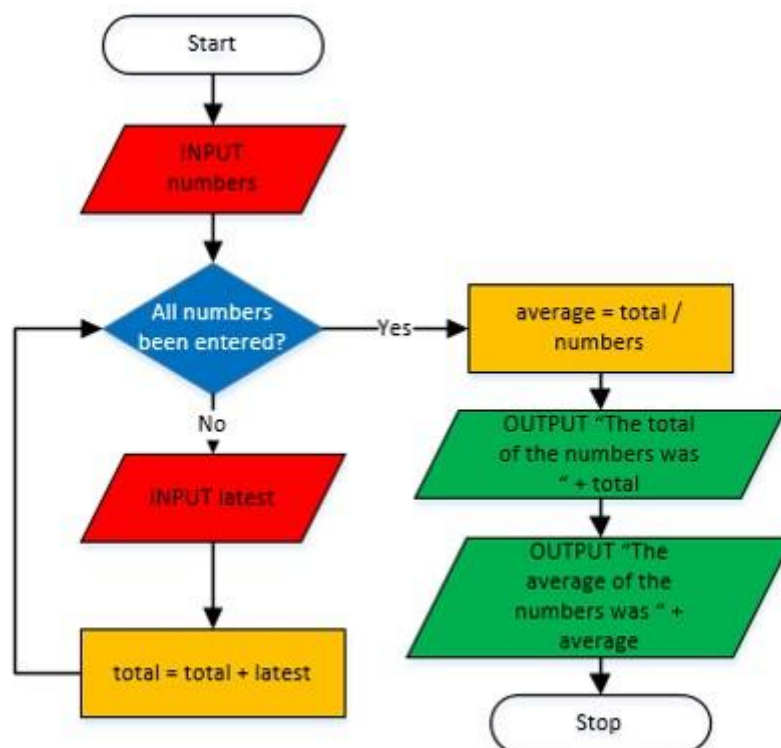
Takrorlanuvchi jarayon — bu dasturda ma'lum buyruqlar ketma-ketligini **bir necha marta qayta bajarish** jarayonidir. Bu jarayon:

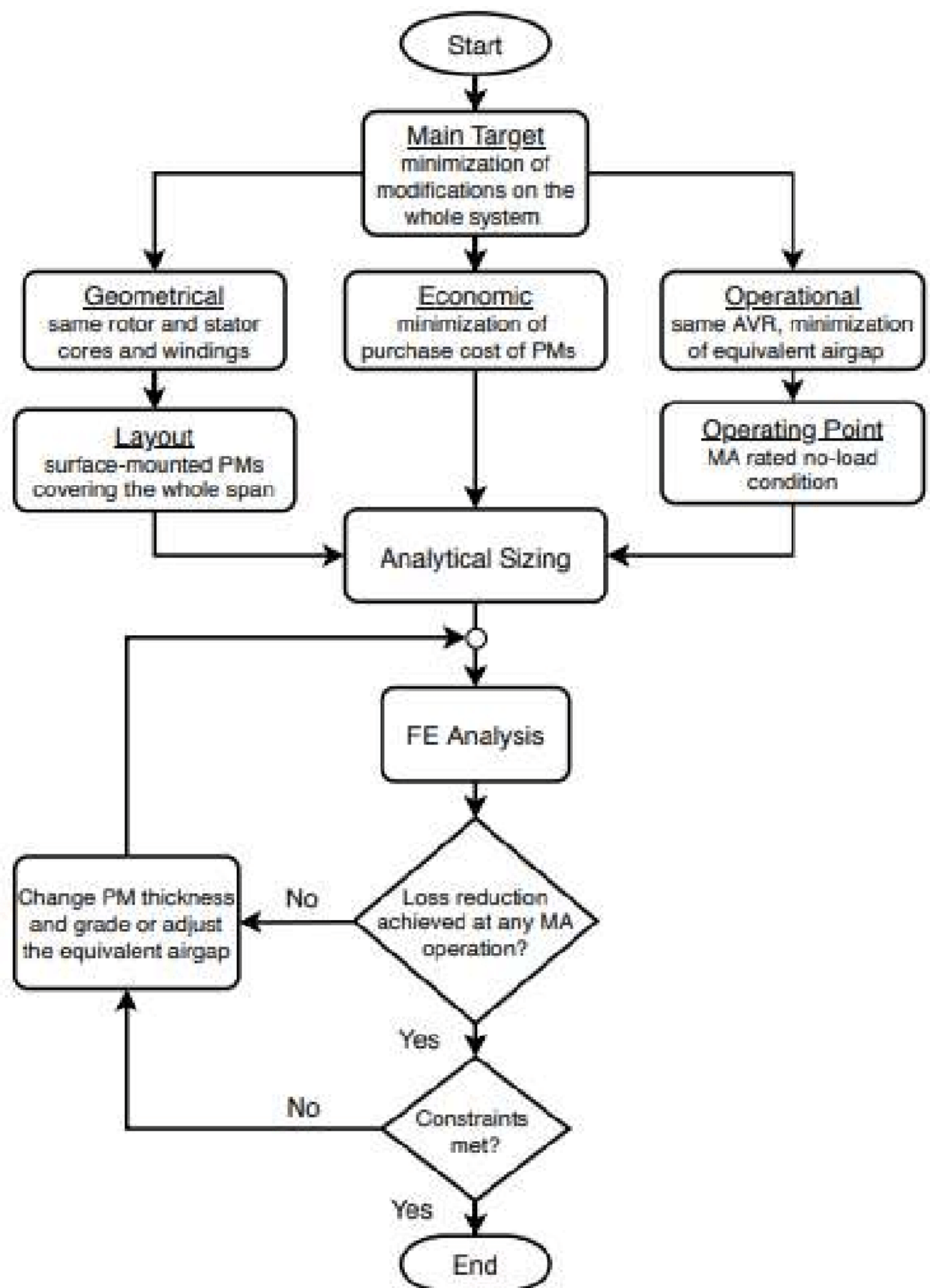
- oldindan ma'lum marta;
- yoki ma'lum shart bajarilgunga qadar davom etishi mumkin.

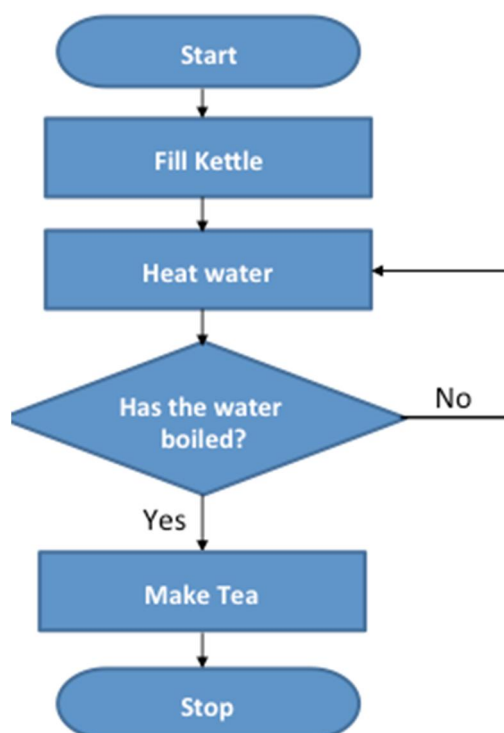
Takrorlanuvchi jarayonlar quyidagi holatlarda ishlatiladi:

- hisoblashlarni avtomatlashtirishda;
- katta hajmdagi ma'lumotlarni qayta ishlashda;
- texnik va muhandislik hisob-kitoblarida;
- algoritmlarni optimallashtirishda.

Python dasturlash tilida takrorlanuvchi jarayonlar asosan **while** va **for** operatorlari yordamida tashkil etiladi.







Yuqoridagi rasmda takrorlanuvchi jarayonning umumiy blok-sxemasi tasvirlangan bo‘lib, shart tekshiriladi va shart rost bo‘lguncha jarayon qayta-qayta bajariladi.

2. Takrorlanuvchi operatorlar turlari, tuzilishi va qo‘llanilishi

Python dasturlash tilida takrorlanuvchi operatorlar **ikki asosiy turga** bo‘linadi:

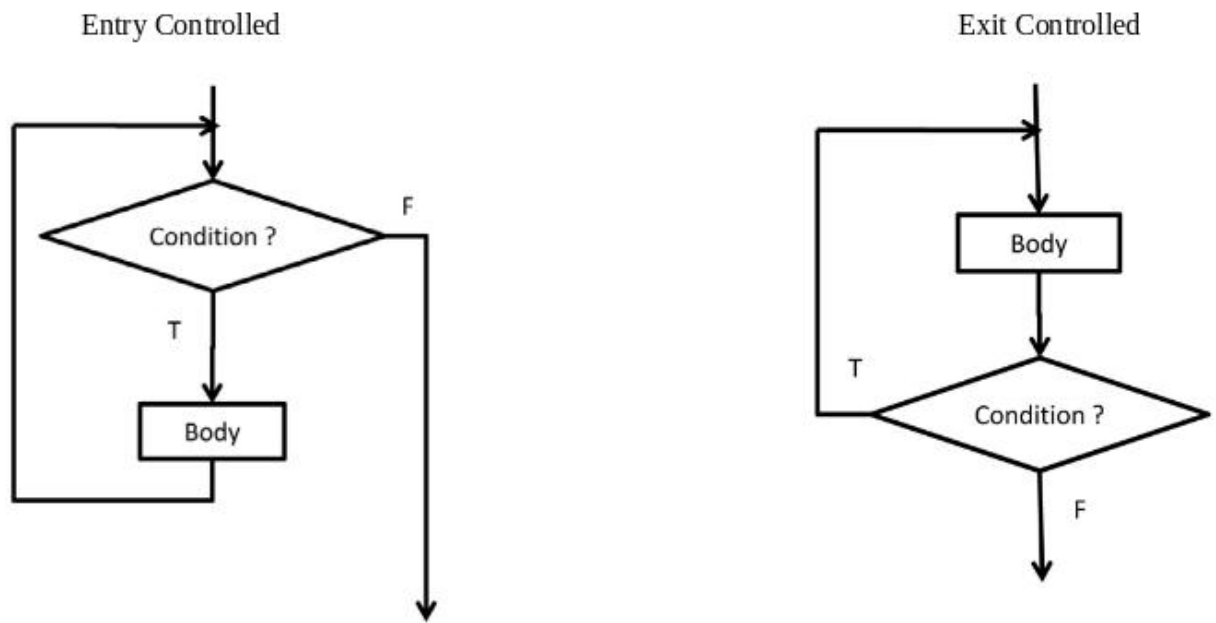
1. **Shartga bog‘liq takrorlanish** (while)
2. **Hisoblagichga bog‘liq takrorlanish** (for)

Har bir operatorning o‘ziga xos tuzilishi va qo‘llanish sohasi mavjud.

Takrorlanuvchi operatorlarning asosiy xususiyatlari:

- buyruqlar blokini bir necha marta bajaradi;
- kodni qisqartiradi;
- dastur mantiqini aniq qiladi;
- xatolik ehtimolini kamaytiradi.

Takrorlanuvchi operatorlar noto‘g‘ri ishlatilsa, **cheksiz sikl** (infinite loop) yuzaga kelishi mumkin. Shu sababli shartlarning to‘g‘ri yozilishi juda muhim.



Entry Controlled Loop (kirishdan boshqariladigan sikl)

Rasmning **chap tomoni** — *Entry Controlled* siklni ifodalaydi.

Mazmuni:

- Avval shart (Condition?) tekshiriladi
- Agar shart True (T) bo'lsa → Body (sikl tanasi) bajariladi
- So'ng yana shart tekshiriladi
- Agar shart False (F) bo'lsa → sikl to'xtaydi

Muhim jihat:

- Sikl tanasi bir marta ham bajarilmasligi mumkin
- Chunki shart boshida tekshiriladi

Dasturlashdagi muqobili:

- while sikli
- for sikli

Python misol:

```
i = 0
while i < 5:
    print(i)
    i += 1
```

Exit Controlled Loop (chiqishdan boshqariladigan sikl)

Rasmning **o'ng tomoni** — *Exit Controlled* siklni ko'rsatadi.

Mazmuni:

- Avval Body (sikl tanasi) bajariladi
- Keyin shart (Condition?) tekshiriladi
- Agar shart True (T) bo'lsa → sikl yana davom etadi
- Agar shart False (F) bo'lsa → sikl tugaydi

Muhim jihat:

- Sikl tanasi kamida bir marta majburiy bajariladi
- Chunki shart oxirida tekshiriladi

Dasturlashdagi muqobili:

- do while sikli (Python'da bevosita yo'q)

Boshqa tillarda (C/C++) misol:

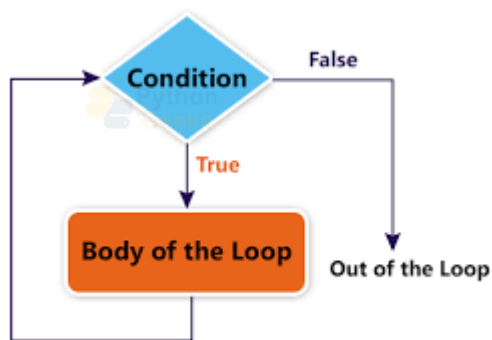
```
do {  
    printf("Hello");  
} while(condition);
```

Python'da taqlid qilish:

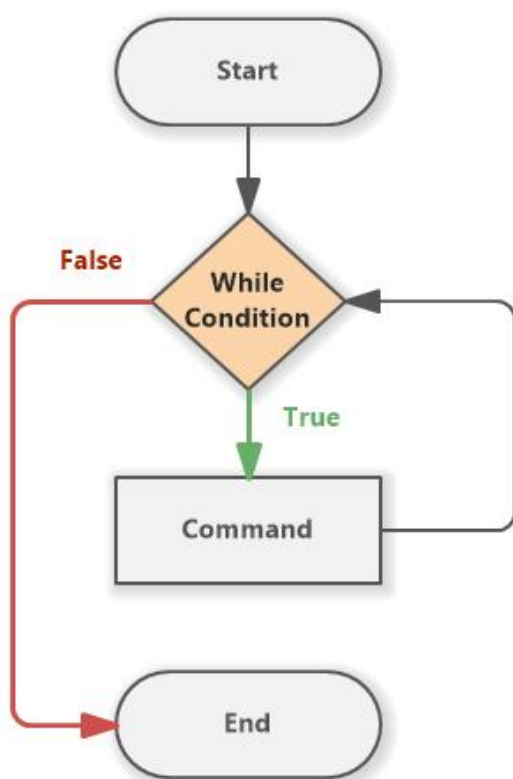
```
while True:  
    print("Hello")  
    if shart == False:  
        break
```

Qisqa taqqoslash

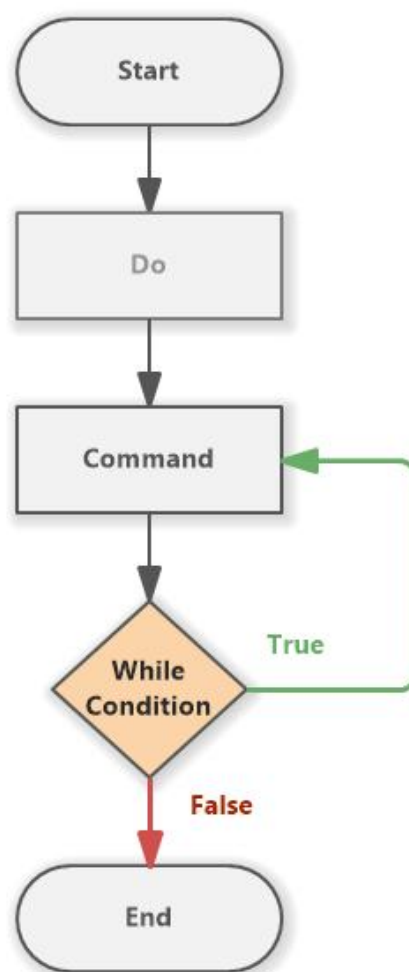
<i>Xususiyat</i>	Entry Controlled	Exit Controlled
<i>Shart qachon tekshiriladi</i>	Boshlanishida	Oxirida
<i>Kamida 1 marta ishlaydimi</i>	Yo'q	Ha
<i>Keng tarqalgan</i>	Ha	Kamroq
<i>Python'da mavjudligi</i>	To'liq	Bevosita yo'q



WHILE



DO-WHILE



Bu rasm while va do-while takrorlanish operatorlarining blok-sxemalarini taqqoslab ko'rsatadi. Ya'ni, sikl qachon shartni tekshirishi va tanasi (command/body) necha marta bajarilishi mumkinligi farqi aniq tasvirlangan.

WHILE sikli (Entry Controlled Loop)

Rasmning **chap tomoni** — while sikli.

Ishlash tartibi:

Dastur **Start** dan boshlanadi

Avval **While Condition** (shart) tekshiriladi

Agar shart **True** bo'lsa → **Command** bajariladi

So'ng yana shartga qaytiladi

Agar shart **False** bo'lsa → **End** (sikl tugaydi)

Muhim jihat:

Shart **sikl boshida** tekshiriladi

Shuning uchun sikl tanasi **umuman bajarilmasligi ham mumkin**

Python misoli:

```
i = 0
```

```
while i < 5:
```

```
    print(i)
```

```
    i += 1
```

DO-WHILE sikli (Exit Controlled Loop)

Rasmning **o'ng tomoni** — do-while sikli.

Ishlash tartibi:

Dastur **Start** dan boshlanadi

Avval **Do / Command** (sikl tanasi) bajariladi

Keyin **While Condition** tekshiriladi

Agar shart **True** bo'lsa → yana tanaga qaytiladi

Agar shart **False** bo'lsa → **End**

Muhim jihat:

Shart **sikl oxirida** tekshiriladi

Sikl tanasi **kamida 1 marta majburiy bajariladi**

C/C++ misoli:

```
do {  
    printf("Hello");  
} while (condition);
```

Python'da to'g'ridan-to'g'ri yo'q, lekin while True + break bilan taqlid qilinadi:

while True:

```
    print("Hello")  
    if condition == False:  
        break
```

3. While operatori va for operatori tuzilishi, dasturda qo'llanilishi

3.1. while operatori

while operatori shartga bog'liq takrorlanishni amalga oshiradi. Sikl shart True bo'lgan vaqt davomida ishlaydi.

while operatorining umumiy tuzilishi:

while shart:
 buyruqlar

Bu yerda:

- shart — mantiqiy ifoda;
- shart rost bo'lsa, sikl davom etadi;
- shart yolg'on bo'lsa, sikl to'xtaydi.

Misol 1: 1 dan 5 gacha sonlarni chiqarish

```
i = 1  
while i <= 5:  
    print(i)  
    i += 1
```

Bu dasturda i o'zgaruvchisi hisoblagich vazifasini bajaradi.

Misol 2: Sonlar yig'indisini hisoblash

```
n = int(input("n ni kiriting: "))
```

```
s = 0
```

```
i = 1
```

```
while i <= n:
```

```
    s += i
```

```
    i += 1
```

```
print("Yig'indi:", s)
```

```
num = 1
```

```
while num<=5:
```

```
    print (num, "John")
```

```
    num = num + 1
```

```
1 John
```

```
2 John
```

```
3 John
```

```
4 John
```

```
5 John
```

YoungWonks

```
x = 15
```

```
y = 0
```

```
while x > 0
```

```
    y = y + 1
```

```
    x = x - y
```

```
endwhile
```

```
print(y)
```

3.2. for operatori

for operatori oldindan ma'lum marta takrorlanadigan jarayonlarda ishlatiladi. Python tilida for operatori asosan range() funksiyasi bilan birga qo'llaniladi.

for operatorining umumiy tuzilishi:

for o'zgaruvchi in ketma-ketlik:

buyruqlar

Misol 1: 1 dan 5 gacha sonlarni chiqarish

```
for i in range(1, 6):  
    print(i)
```

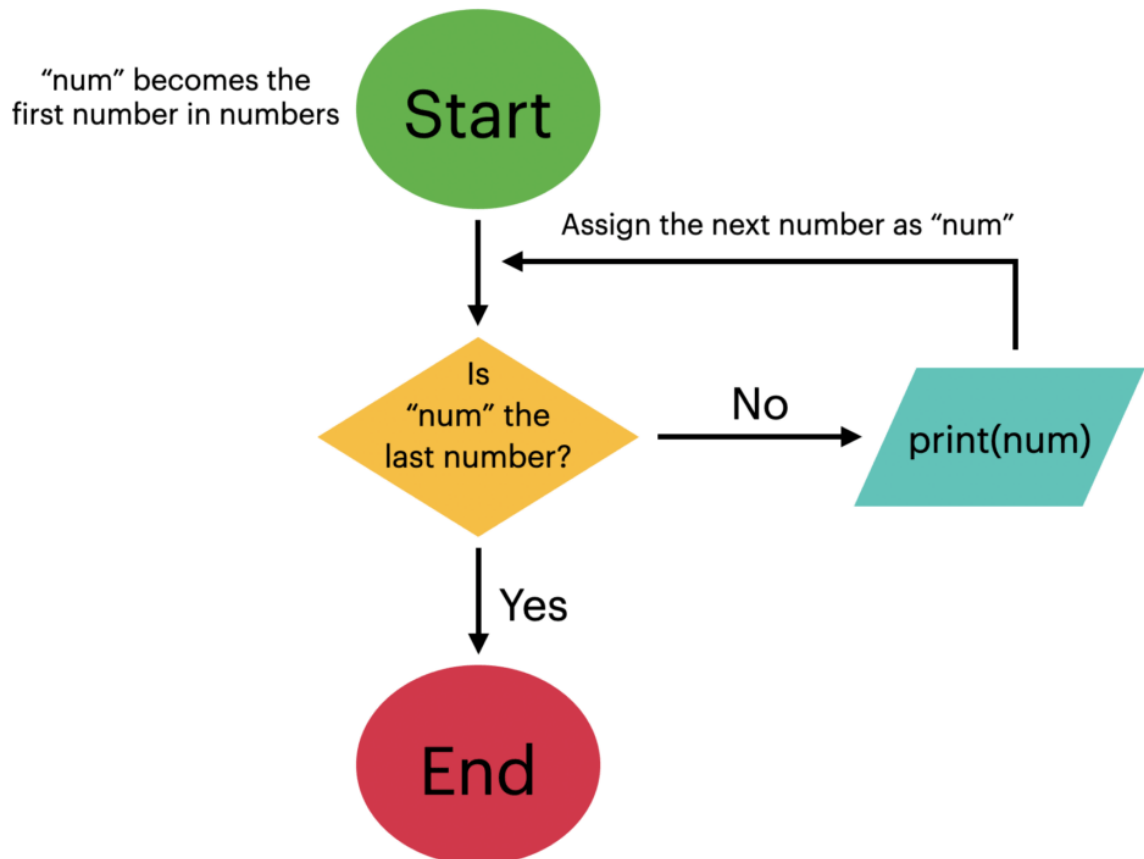
Misol 2: Sonlar yig'indisini hisoblash

```
n = int(input("n ni kiriting: "))  
s = 0
```

```
for i in range(1, n + 1):  
    s += i
```

```
print("Yig'indi:", s)
```

numbers = [1, 2, 3, 4, 5]



```

In [10]: number = 5

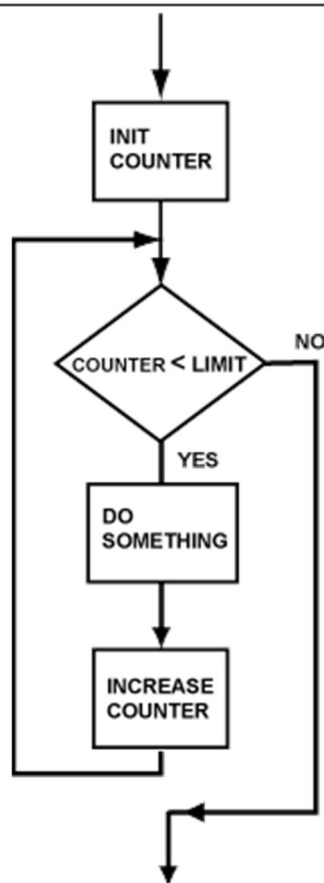
if number > 1:
    # check for factors
    for i in range(2,number):
        if (number % i) == 0:
            print(number,"is not a prime number")
            break

    else:
        print(number,"is a prime number")

# if input number is less than
# or equal to 1, it is not prime
else:
    print(num,"is not a prime number")

5 is a prime number

```



while va for operatorlarini taqqoslash

Xususiyat	while	for
Shart	Oldindan noma'lum	Oldindan ma'lum
Hisoblagich	Qo'lda boshqariladi	Avtomatik
Qo'llanish	Murakkab shartlar	Hisoblashlar

4. Takrorlanuvchi jarayonlarga dasturlar tuzish

Takrorlanuvchi jarayonlarga dastur tuzishda quyidagi bosqichlar amalga oshiriladi:

1. masalani tahlil qilish;
2. takrorlanadigan amallarni aniqlash;
3. sikl turini tanlash (while yoki for);
4. shart yoki chegarani belgilash;
5. Python dasturini yozish va sinash.

4.1. Amaliy misol (texnik yoʻnalish)

Masala: 10 ta oʻlchov natijasining oʻrtacha qiymatini hisoblash.

```
s = 0
for i in range(10):
    x = float(input("Qiymatni kiriting: "))
    s += x
```

```
orta = s / 10
print("Oʻrtacha qiymat:", orta)
```

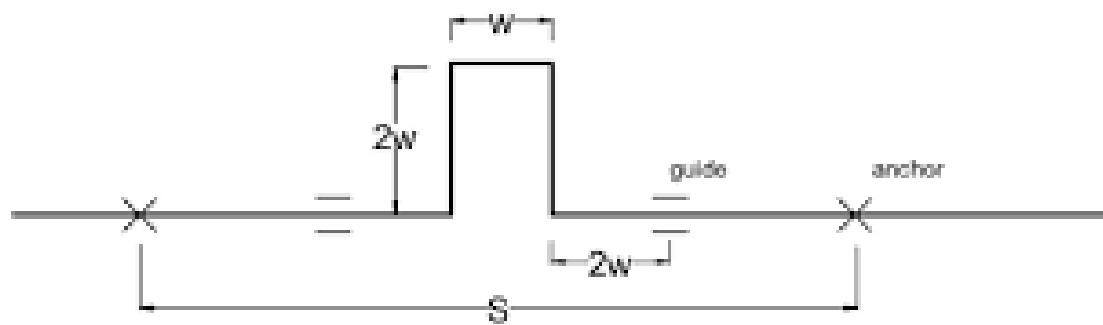
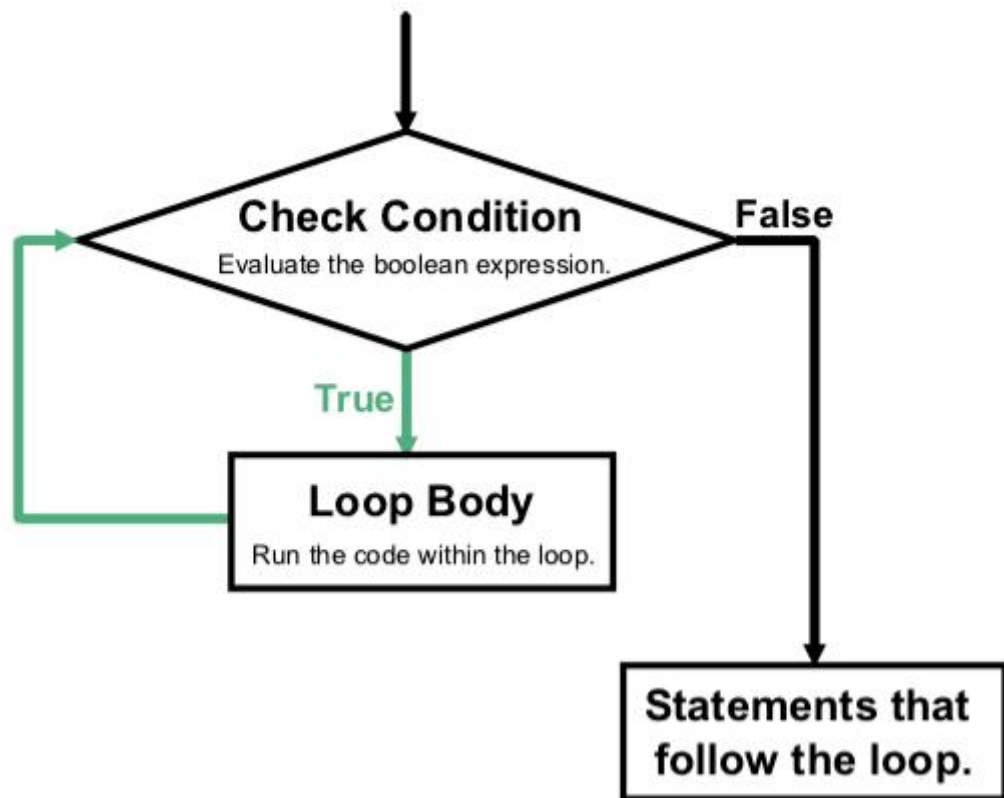
4.2. While operatoriga oid amaliy misol

Masala: Foydalanuvchi 0 kiritmaguncha sonlarni qabul qilish va ularning yigʻindisini topish.

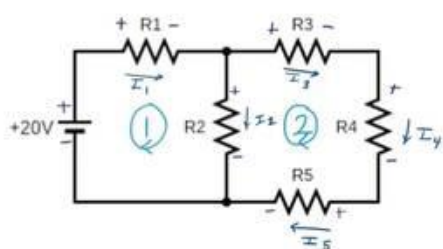
```
s = 0
son = int(input("Son kiriting (0 - tugatish): "))

while son != 0:
    s += son
    son = int(input("Son kiriting (0 - tugatish): "))

print("Yigʻindi:", s)
```



Ex 1: Write the loop rule for each loop in the circuit shown below.



$$\sum V_{loop} = 0$$

$$V = IR$$

Loop 1

$$+20 - V_1 - V_2 = 0$$

$$+20 - (I_1 R_1) - (I_2 R_2) = 0$$

Loop 2

$$-V_3 - V_4 - V_5 + V_2 = 0$$

$$-(I_3 R_3) - (I_4 R_4) - (I_5 R_5) + (I_2 R_2) = 0$$

© Study.com

Xulosa

Takrorlanuvchi jarayonlar dasturlashning eng muhim tushunchalaridan biridir. Python dasturlash tilida while va for operatorlari yordamida murakkab va katta hajmdagi hisob-kitoblarni juda sodda va samarali tarzda bajarish mumkin. Ushbu mavzu talabalarni real hayotiy va texnik masalalarni dasturiy yechishga tayyorlaydi hamda keyingi mavzu — **break va continue operatorlarini** o'rganish uchun mustahkam asos yaratadi.

Nazorat savollari

1. Takrorlanuvchi jarayon deb nimaga aytiladi?
2. Python tilida takrorlanuvchi operatorlar qaysilar?
3. while operatori qanday ishlaydi?
4. for operatorining vazifasi nimadan iborat?
5. while va for operatorlarining farqi nimada?
6. Cheksiz sikl nima va u qanday yuzaga keladi?
7. range() funksiyasi qanday ishlaydi?
8. Takrorlanuvchi jarayonlarda hisoblagichning roli nimada?
9. Texnik masalalarda takrorlanuvchi jarayonlarning ahamiyati nimada?
10. Takrorlanuvchi dastur tuzish bosqichlarini sanab bering.